GUYUB: Technical Proposal

All-in-One Operations Platform for Community & Campus Event Orchestration

Project Name	Guyub (SaaS / PaaS)	**Hackathon Track**	Case 2: Stronger Communities
Target Audience	Ormawa, Communities, SMEs	**Development Period**	1-Week Sprint (Hackathon Target)

1. EXECUTIVE SUMMARY & PROBLEM STATEMENT

Student organizations (Ormawa) and local communities regularly face severe operational inefficiencies when planning and executing events. Key pain points include unstructured sponsorship outreach, manual spreadsheet-based attendee tracking, chaotic physical check-in queues, and high post-event administrative burden (manual certificate generation and delivery). Guyub solves this by unifying event management into an automated, role-based platform.

2. MULTI-ROLE OPERATING MODEL & MONETIZATION

Platform Access Roles

- **Guyub Admin:** Platform monitoring, tenant security, and system metrics.
- **Event Organizer:** Event setup, AI proposal generator, certificate builder.
- **Committee Member:** Dedicated mobile/web QR scanner for fast check-in.
- **General Attendee:** Dynamic QR ticket portal & instant credential wallet.

Tiered Business Model

- **Free Tier:** Free for campus student organizations (Ormawa) & micro-events.
- **Event Pass (IDR 149k):** Pay-per-event for mid-sized community summits.
- **Subscription (5%–8%):** Transaction commission model for paid ticketing events.

3. HYBRID TECHNICAL ARCHITECTURE

To balance striking visual marketing with high-performance operational dashboards, Guyub adopts a modern **Hybrid Frontend + Serverless Backend** architecture.

Architecture Layer	Technology Choice	Strategic Rationale
Public Landing Page	Next.js + TypeScript	High SEO ranking, rich WebGL/Framer Motion animations, fast page loads.
Dashboards (All Roles)	Flutter Web	Unified cross-platform codebase across web and mobile; real-time UI state.
Authentication & Security	Firebase Auth	Role-Based Access Control (RBAC) enforcing custom claims.
Database & Real-time Sync	Cloud Firestore	NoSQL real-time listeners for instant attendance sync across scanner devices.
Backend Automations	Google Cloud Functions	Node.js background triggers for instant certificate generation & email delivery.

Architecture Layer	Technology Choice	Strategic Rationale
Asset Storage	Firebase Storage	Secure storage for event banners, sponsor logos, and PDF certificate templates.

4. FIRESTORE DATA MODEL SPECIFICATION

```
collections/ ├── users/{uid} | ├── role: "admin" | "organizer" | "committee" | "attendee" | └── organization_id: string ├── events/{eventId} | ├── title: string, date: timestamp, status: "draft" | "active" | "completed" | └── certificate_template_url: string └── attendance/{attendanceId} ├── event_id: string, user_id: string, qr_hash: string ├── is_checked_in: boolean, checked_in_at: timestamp └── certificate_issued: boolean
```

5. IMPLEMENTATION PLAN (1-WEEK HACKATHON ROADMAP)

Days 1–2: Setup Next.js landing page & Flutter Web navigation routing. Configure Firebase Auth & Firestore Security Rules.
Days 3–4: Build Organizer Dashboard (Event creation + AI pitch helper) and Committee QR Scanner interface.
Days 5–6: Implement Google Cloud Functions for auto-generating personalized PDF certificates upon check-in.
Day 7: End-to-end testing, UI polish, production deployment via Firebase Hosting & Vercel.